

Class

- Class એ template છે જે object ની attributes અને behavior ને દર્શાવે છે.
- Class એ બ્લુપ્રિન્ટ અથવા પ્રોટોટાઇપ છે જેમાંથી ઘણા objects ને બનાવી શકાય છે.
- Class એ abstract data type (ADT) નું implementation છે. જે attributes અને methods ને define કરે છે તેમજ ADT ની data structure અને operations ને implement કરે છે.

Syntax:

```
class classname
{
    Datatype variable1;
    Datatype variable2;
    // ...
    Datatype variableN;

    return_type methodname1(parameter-list)
    {
        // body of method
    }

    return_type methodname2(parameter-list)
    {
        // body of method
    }

    return_type methodnameN(parameter-list)
    {
        // body of method
    }
}
```

- class ને class keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે.
- class માં declare કરેલા ડેટા અને વેરીએબલ્સને ઇન્સ્ટન્સ વેરીએબલ્સ કહેવામાં આવે છે કારણ કે class ના દરેક object માટે આ variables ની પોતાની copy હોય છે તેથી એક object નો ડેટા બીજા object ના ડેટાથી અલગ અને unique હોય છે.
- methods ની અંદર કોડ ને લખવામાં આવે છે.
- class માં defined કરેલ methods અને variables એ class ના members છે.

Methods

Syntax:

```
return_type method_name(parameter-list)
{
    // body of method
}
```

}

- અહીં return_type એ method માંથી return થયેલ ડેટાની type દર્શાવે છે. તે કોઈપણ data type નો હોઈ શકે છે. જો method કોઈ value return કરતી નથી, તો તેનો રીટર્ન type void હોય છે.
- method_name એ methodનું નામ દર્શાવે છે. આ કોઈપણ valid identifier હોયશકે જે current scope માં પહેલાથી declare કરેલ ન હોવા જોઈએ. parameter-list એ type અને identifier ની comma separate કરેલ sequence છે.
- Parameters એ variables છે. જે method call કરવામાં આવે ત્યારે argument તરીકે pass કરવામાં આવે છે. જો method માં કોઈપણ parameters ન હોય તો parameter list ને empty રાખવામાં આવે છે.

Return value

- જ્યારે method માં void સિવાય કોઈપણ return type હોય ત્યારે calling method ને નીચે આપેલ form દ્વારા value return કરવામાં આવે છે.

Syntax:

return value;

- અહીં, value એ return કરવાની હોય તેને દર્શાવે છે.

Example:

```
class Box
{
    double width = 1;
    double height = 2;
    double depth = 3;

    double volume()
    {
        return (width * height * depth);
    }
}
```

Method Call

Syntax:

var_name = **object_name**.method_name(parameter-list);

Example:

vol = b1.volume();

- ઉપરના ઉદાહરણમાં, b1 એ એક object છે અને જ્યારે volume() ને call કરવામાં આવે છે, ત્યારે તે એસાઈનમેન્ટ સ્ટેટમેન્ટની જમણી બાજુ મૂકવામાં આવે છે. ડાબી બાજુ માં variable હોય છે અહીં vol માં volume() દ્વારા return થયેલ value store થાય છે.

Declaring Object

- class નો object બનાવાવા માટે class type નો variable declare કરવામાં આવે છે અને આ variable કોઈ object ને define કરતો નથી. તે ફક્ત variable જ છે જે કોઈ object ને refer કરી શકે છે.
- new operator નો use કરીને object ની original copy લેવામાં આવે છે અને variable ને assign કરવામાં આવે છે.

- new operator એ object માટે dynamically memory allocate કરે છે અને allocated કરેલી memory નું address આપે છે.

Syntax:

Step - 1 : class_name class_var;

Step - 2: class_var = new class_name();

- અહીં, class_var એ class type નો variable create બનાવામાં આવે છે. class_name એ class નું નામ છે જે ના object બનાવામાં આવે છે.

Assigning Object Reference Variables

- Object ના રેફરન્સ વેરીએબલ્સ ને assignment operator સાથે ઉપયોગ કરવામાં આવે ત્યારે તે જુદી રીતે act કરે છે.

Example:

```
Box b1 = new Box();
```

```
Box b2 = b1;
```

- અહીં b1 અને b2 બંને એક જ object ને refer કરે છે. જ્યારે b1 થી b2 માં assignment થાય ત્યારે કોઈ પણ memory allocate થતી નથી.
- તેથી b2 દ્વારા કોઈપણ changes કરવામાં આવે તો તે b1 ને effect કરશે કારણકે b2 અને b1 બંને same object ને refer કરે છે.

Example:

```
class Box
{
    double width = 1.0;
    double height = 2.0;
    double depth = 3.0;

    void volume()
    {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}

class BoxDemo
{
    public static void main(String args[])
    {
        Box b1 ;           // declare reference to object
        b1 = new Box()      // allocate a Box object
        //Box b1 = new Box(); // We can also combine above two statement.
        b1.volume();
    }
}
```

Output:

Volume is 6.0

Visibility Controls of JAVA

- classes, variables, methods અને constructors ની access level ને કન્ટ્રોલ કરવા માટે java માં જુદા જુદા access modifier આવેલ છે. JAVA માં four access levels છે:
 - 1) **Package/friendly (default)** – પેકેજ માં access થાય છે. કોઈ modifiers ની જરૂર નથી.
 - 2) **Private** – class માં જ access થાય છે.
 - 3) **Public**- class માં અને class ની બહારના part માં પણ access થાય છે.
 - 4) **Protected**- પેકેજ માં અને બધા sub class માં access થાય છે.

Default Access Modifier – No Keyword

- Default access modifier નો meaning એ છે કે class, field, method માટે access modifier declare કરવામાં આવતા નથી.
- કોઈપણ access control મોડિફાયર નો use કર્યા વગર declare કરેલ variable અથવા method ને same package માં રહેલા બીજા class access કરી શકે છે. ઇન્ટરફેસમાં methods માટે ડિફોલ્ટ મોડિફાયરનો ઉપયોગ કરી શકાતો નથી કારણ કે (interface) ઇન્ટરફેસ માં રહેલી methods by default public હોય છે.

Example:

```
String str = "Hi";  
void a()  
{  
    System.out.println(str);  
}
```

Private Access Modifier – private

- private declare કરેલ Methods, Variables અને Constructors ને જે class માં declare કરેલા હોય તે જ classમાં જ access કરી શકાય છે.
- Private access modifier એ સૌથી વધારે restrictive (રિસ્ટ્રિક્ટિવ) access level આપે છે. Class and interfaces એ private હોઈ શકે નહિ.
- જો public getter methods class માં હોય તો private declare કરાયેલ Variables ને class ની બહાર access કરી શકાય છે.
- private મોડિફાયરનો ઉપયોગ કરીને object encapsulate થાય છે અને બહારના world થી ડેટા ને hide કરે છે.

Example:

```
class A  
{  
    private String s1 = "Hello";  
    public String getName()  
    {  
        return this.s1;  
    }  
}
```

- અહીં, A class નો s1 variable private છે, તેથી બીજા class માં access થઈ શકે નહિ. જો આ variable ને class ની બહાર access કરવો હોય તો getName() નામ ની public method declare કરેલ છે જે s1 ની value return કરે છે.

Public Access Modifier - public

- Public keyword એ access specifier છે જે પ્રોગ્રામરને class ના members ની visibility ને કન્ટ્રોલ કરવા માં મદદ કરે છે.
- જ્યારે class ના member ને public declare કરેલ હોય ત્યારે તે class ની બહાર access થઈ શકે છે.
- Class, method, constructor અને interface વગેરે ને public declares કરેલ હોય તો તેને બીજા class દ્વારા access કરી શકાય છે.
- જેથી Public class માં declare કરેલ methods અથવા blocks ને Java world માં આવેલ કોઈપણ class દ્વારા access કરી શકાય છે.
- જો આપણે કોઈબીજા package માં રહેલા class ને access કરવો હોય તો તેને import કરવો પડે છે કારણકે Class માં આવેલ public methods અને variables ને inheritance નો use કરી ને sub class દ્વારા inherit કરવામાં આવે છે.

Example:

```
public static void main(String[] args)
{
    // ...
}
```

- એપ્લિકેશનની main () method public હોવી જરૂરી છે. નહિતર, જાવા ઇન્ટરપ્રીટર main method ને call કરી શકે નહિ.

Protected Access Modifier – Protected

- Variables, methods અને constructors ને super class માં protected declare કરવામાં આવ્યા છે હોય તેને ફક્ત બીજા પેકેજ અથવા same પેકેજ માં રહેલ sub class માં access કરી શકાય છે.
- Protected access મોડિફાયર class અને interfaces સાથે use કરી શકાય નહિ. Methods ને protected declare કરી શકાય છે. જ્યારે ઇન્ટરફેસમાં રહેલ Methods ને protected declare કરી શકાતી નથી.
- Protected access મોડિફાયર એ subClass ને super class ની helper method અથવા variable use કરવા દે છે જ્યારે non-related class ને use કરતા અટકાવે છે.

Example:

```
package p1;
class A
{
    float f1;
    protected int i1;
}
```

// note that class B belongs to the same package as class A

```
package p1;
class B
```

```
{
    public void getData()
    {
        // create an instance of class A
        A a1 = new A();
        a1.f1 = 19;
        a1.i1 = 12;
    }
}
```

- ઉપરના ઉદાહરણમાં, class A અને B same પેકેજ p1 માં છે. Class A નો i1 variable છે જે protected તરીકે declare કરેલ છે. તેથી, તે સંપૂર્ણ પેકેજ અને તેના subclasses દ્વારા access કરી શકાય છે. આમ, class B ની getData() methodમાં આપણે variable f1 અને આઈ i1 નો ઉપયોગ કરી શકીએ છીએ.

this Keyword

- ક્યારેક કોઈ method જે object દ્વારા invoke થયેલ હોય તેને refer કરવાની જરૂર પડે છે.
- તેના માટે this keyword નો use થાય છે. this keyword ને method અને constructor માં use કરી શકાય છે જે current object ને refer કરે છે.
- તેનો અર્થ એ કે, this હંમેશા તે object refer કરે છે કે જેનો use કરીને તે method call કરવામાં આવેલ હોય.
- Variable hide થતા હોય તે case માં this keyword ખૂબ ઉપયોગી થઈ શકે છે.
- class ના current object ને refer કરવામાટે કોઈપણ જગ્યાએ use કરી શકાય છે.
- એક જ નામ ના બે Instance/Local variables બનાવી શક્ય નહિ. પરંતુ એક જ નામ ના instance variable અને local variable અથવા method parameter declare કરી શકાય છે.
- લોકલ વેરીએબલ એ ઈન્સ્ટન્સ વેરિએબલને hide કરે છે જેને Variable Hiding કહે છે.

Example:

```
class A
{
    int v = 5;
    public static void main(String args[])
    {
        A a1 = new A();
        a1.method(20);
        a1.method();
    }
    void method(int variable)
    {
        int v = 10;
        System.out.println("Value of Instance variable :" + this.v);
        System.out.println("Value of Local variable :" + v);
    }
    void method()
    {
        int v = 40;
        System.out.println("Value of Instance variable :" + this.v);
        System.out.println("Value of Local variable :" + v);
    }
}
```

```
}
```

Output:

Value of Instance variable :5

Value of Local variable :10

Value of Instance variable :5

Value of Local variable :40

static Keyword

- જ્યારે member ને static declare કરવામાં આવે છે, ત્યારે તેના class ના કોઈપણ objects બનાવવામાં આવે તે પહેલાં, અને કોઈપણ objects ને refer કરે તે પહેલાં.
- methods અને variablesને static declare કરી શકાય. static method નું સૌથી best ઉદાહરણ main() method છે.
- main() ને static declare કરવામાં આવે છે કારણ કે કોઈપણ object create થાય તે પહેલાં તેને call કરવામાં આવે છે.
- જ્યારે class નો object create કરવામાં આવે ત્યારે static variable ની new copy બનાવવામાં આવતી નથી. બધા class same static variable ને share કરે છે.
- static method માં નીચે મુજબ restriction(રિસ્ટ્રિક્શન) છે:
 - 1) તે બીજા static method ને જ call કરી શકે છે.
 - 2) તે static data ને જ access કરી શકે છે.
 - 3) તે માં this અને super keyword ને ઉપયોગ થઈ શકે નહિ.

Example:

```
class staticDemo
{
    static int count=0;    //will get memory only once and retain its value

    staticDemo()
    {
        count++;
        System.out.println(count);
    }
    static
    {
        System.out.println("Static block initialized...");
    }
    static void display()
    {
        System.out.println("Static method call...");
    }
    public static void main(String args[])
    {
        staticDemo s1=new staticDemo();
        staticDemo s2=new staticDemo();
    }
}
```

```
staticDemo s3=new staticDemo();
display();
}
```

Output:

```
Static block initialized...
1
2
3
Static method call...
```

- જો class ની બહારથી static methodને call કરવી હોય તો તે નીચે મુજબ કરી શકાય:
classname.method();
- અહીં, classname એ class નું નામ છે જેમાં static method ને declare કરેલ છે. Static method ને class ના object દ્વારા call કરવાની જરૂર નથી.

final Keyword

- variable ને final declare કરી શકાય છે. final variable ની value ને બદલી શકાય નહિ. તેનો અર્થ એ છે કે variable એ constant declare થશે જેની value change ન થઈ શકે.
- જો કોઈપણ method ને final declare કરવામાં આવેલ હોય તો તેને ઓવરરાઈડ ન કરી શકાય.
- જો તમે કોઈપણ class ને final declare કરેલ હોય તો તેને inherit ન કરી શકાય.

Example:

```
class finalDemo
{
    final int b = 100;
    void m1()
    {
        b = 200; /* Error generate because we cannot change the value of
                  final variable*/
    }
    public static void main(String args[])
    {
        finalDemo f1 = new finalDemo();
        f1.m1();
    }
}
```

Output:

Compile Time Error

Method Overloading

- જો class માં એક જ નામની અને જુદા જુદા parameters સાથે ઘણી methods હોય તો તેને મેથડ ઓવરલોડિંગ તરીકે ઓળખવામાં આવે છે.
- મેથડ ઓવરલોડિંગને કમ્પાઈલ ટાઈમ (સ્ટેટિક) પોલીમોર્ફિઝમ તરીકે પણ ઓળખવામાં આવે છે.

- એક જ નામ ની method માં જુદા જુદા parameters અને જુદી જુદી data type ના parameters નો ઉપયોગ કરવામાં આવે છે.
- જુદી જુદી return types સાથે method નું ઓવરલોડિંગ થઈ શકે નહિ.
- એક જ નામ થી declare થયેલ એક કરતા વધારે method માંથી કઈ method call કરવી તે parameters ની data type અને parameters ની સંખ્યા પર આધાર રાખે છે.
- એક જ નામ ની બે methods નીચે આપેલ કોઈ એક point થી different હોવી જોઈએ,
 - 1) parameters ની સંખ્યા
 - 2) parameters ની ડેટા type
 - 3) parameter નો order

Example:

```
class overloadingDemo
{
    void sum(int a,int b)
    {
        System.out.println("Sum of (a+b) is:: "+(a+b));
    }
    void sum(int a,int b,int c)
    {
        System.out.println("Sum of (a+b+c) is:: "+(a+b+c));
    }
    void sum(double a,double b)
    {
        System.out.println("Sum of double (a+b) is:: "+(a+b));
    }
    public static void main(String args[])
    {
        overloadingDemo o1 = new overloadingDemo();
        o1.sum(10,10);        // call method1
        o1.sum(10,10,10);    // call method2
        o1.sum(10.5,10.5);   // call method3
    }
}
```

Output:

```
Sum of (a+b) is:: 20
Sum of (a+b+c) is:: 30
Sum of double (a+b) is:: 21.0
```

Constructor

- કન્સ્ટ્રક્ટર એ special method છે જેનો ઉપયોગ object ને initialize કરવા માટે થાય છે.
- તે object બનાવતી વખતે automatic call થાય છે.
- કન્સ્ટ્રક્ટરને define કરવા માટેના નીચે પ્રમાણે બે નિયમો છે:

- 1) કન્સ્ટ્રક્ટર નું નામ અને class નું નામ સરખું જ હોવું જોઈએ.
- 2) કન્સ્ટ્રક્ટરને કોઈ return type હોતી નથી.
- class constructor ની રીટર્ન type એ class type હોય છે.
- **constructor ની બે type છે.**
 - 1) Default Constructor
 - 2) Parameterized constructor

Default Constructor

- જે કન્સ્ટ્રક્ટર માં એકપણ પેરામીટર ન હોય તે કન્સ્ટ્રક્ટર ને default કન્સ્ટ્રક્ટર તરીકે ઓળખવામાં આવે છે
- જો એકપણ class માં કન્સ્ટ્રક્ટર ન બનાવામાં આવે તો compiler દ્વારા તે class માટે default કન્સ્ટ્રક્ટર બનાવે છે.

Example:

```
class A
{
    A()                //Default constructor
    {
        System.out.println("Default constructor called..");
    }
    public static void main(String args[])
    {
        A a = new A();
    }
}
```

Output:

Default constructor called..

Parameterized Constructor

- જે કન્સ્ટ્રક્ટર માં પેરામીટર હોય છે તે કન્સ્ટ્રક્ટર ને Parameterized કન્સ્ટ્રક્ટર તરીકે ઓળખવામાં આવે છે
- તેનો ઉપયોગ જુદા જુદા object ને જુદી જુદી values આપવા માટે થાય છે.
- objects બનાવામાં આવે ત્યારે parameter પાસ કરવા જરૂરી છે.
- જો ફક્ત પેરામીટરોનું કન્સ્ટ્રક્ટરને જ define કરેલ હોય, તો default constructor થી object બનાવી શકાય નહીં. કારણ કે કમ્પાઈલર default constructor બનાવશે નહીં પરંતુ આપણે default constructor ને બનાવવું પડશે.

Example:

```
class A
{
    int a;
    String s1;
    A(int b,String s2)    //Parameterized constructor
    {
        b = a;
        s2 = s1;
    }
    void display()
    {
        System.out.println("Value of parameterized constructor is :: "+a+" and
```

```

        "+b);
    }
    public static void main(String args[])
    {
        A a = new A(10,"Hello");
        a.display();
    }
}

```

Output:

Value of parameterized constructor is :: 10 and Hello

Copy Constructor

- copy કન્સ્ટ્રક્ટર એક કન્સ્ટ્રક્ટર છે જે માં ફક્ત એક જ parameter pass કરવામાં આવે છે અને તે જ class માં કન્સ્ટ્રક્ટર define કરેલ હોય તે class ની type નો હોય છે.
- Copy constructorનો ઉપયોગ parameter તરીકે પાસ કરેલા object ની new copy બનાવવા માટે થાય છે પરંતુ new copy એ original object થી independent હોય છે.
- તે independent નો meaning એ છે કે નવા object નું memory location original object ની memory location થી અલગ હોય છે.

Overloading Constructor

- એક class ની માં એક કરતા વધારે Constructor define કરી શકાય છે તેને Constructor overloading તરીકે ઓળખવામાં આવે છે.
- તે મેથડ ઓવરલોડિંગ જેવુંજ છે. કન્સ્ટ્રક્ટર ઓવરલોડિંગમાં તમારી પાસે different signature સાથે multiple constructors define કરેલા હોય છે અને difference એ છે કે કન્સ્ટ્રક્ટર પાસે રીટર્ન type હોતી નથી.
- આ types ના કન્સ્ટ્રક્ટર ને overloaded constructor તરીકે ઓળખાય છે.

Passing object as a parameter

- જો કોઈ નવો object બનાવવો હોય જે existing object જેવો જ હોય ત્યારે class માં constructor બનાવવામાં આવે છે જેમાં paramater તરીકે class નો object pass કરવામાં આવે છે.

Example:

```

class Box
{
    double width;
    double height;
    double depth;
    // It takes an object of type Box. Copy constructor
    Box(Box ob)
    {
        // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
}

```

```
// Parameterized constructor
Box(double w, double h, double d)
{
    width = w;
    height = h;
    depth = d;
}
// Default constructor
Box()
{
    width = -1; // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1; // box
}
// constructor used when cube is created
Box(double len)
{
    width = height = depth = len;
}
// compute and return volume
double volume()
{
    return width * height * depth;
}
}
class DemoAllCons
{
    public static void main(String args[])
    {
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        Box myclone = new Box(mybox1); // create copy of mybox1
        double vol;
        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        // get volume of cube
        vol = mycube.volume();
    }
}
```

```
        System.out.println("Volume of cube is " + vol);  
        // get volume of clone  
        vol = myclone.volume();  
        System.out.println("Volume of clone is " + vol);  
    }  
}
```

Output:

```
Volume of mybox1 is 3000.0  
Volume of mybox2 is -1.0  
Volume of cube is 343.0  
Volume of clone is 3000.0
```